

RAG Implementation Plan — Al-Furqan

Tafsir KB

Engine-Guided Reasoning with Retrieval-Augmented Generation

Version 1.0

Project: Al-Furqan (الفرقان)

Date: March 22, 2026

Status: Draft — Pending Review

Author: Arif AI + Muhammad Al-Ashmawy

Classification: Internal — Variance R&D

1. الرؤية

الهدف الأساسي:

مش بس يجيب معلومات — تعليم المودل يفكر أحسن.

طريقة الشيخ أحمد — **منهج تفكير** هو KB مش مخزن بيانات بنسحب منه إجابات جاهزة. ال KB ال
يعلم المودل Engine السيد في ربط الآيات ببعضها وبالسيرة والأحاديث والسنن الإلهية. هدفنا إن ال
نفس طريقة التفكير دي.

الفلسفة:

المعلم (يوجه التفكير ويتحقق من النتيجة) = ال Engine
المصادر + أنماط التفكير المستخرجة من الشيخ = ال KB
الطالب (يفكر ويجاوب بتوجيه المعلم) = ال LLM

المودل - (الطالب يحفظ مش يفهم — Pipeline RAG) يفكر بدل المودل Engine ال - : **مش**
(الطالب يبذاكر من غير منهج — Agentic RAG) يفكر لوحده

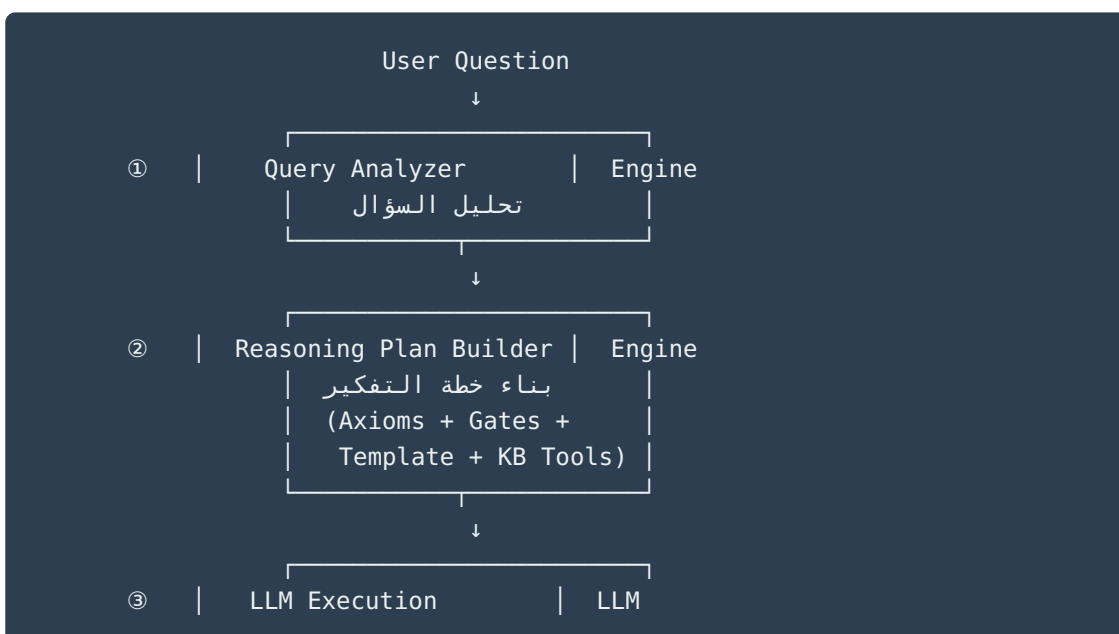
(Engine-Guided Reasoning) يوجّه تفكير المودل بالمصادر الصح + أسئلة التفكير الصح Engine ال - : **بل**
(Reasoning)

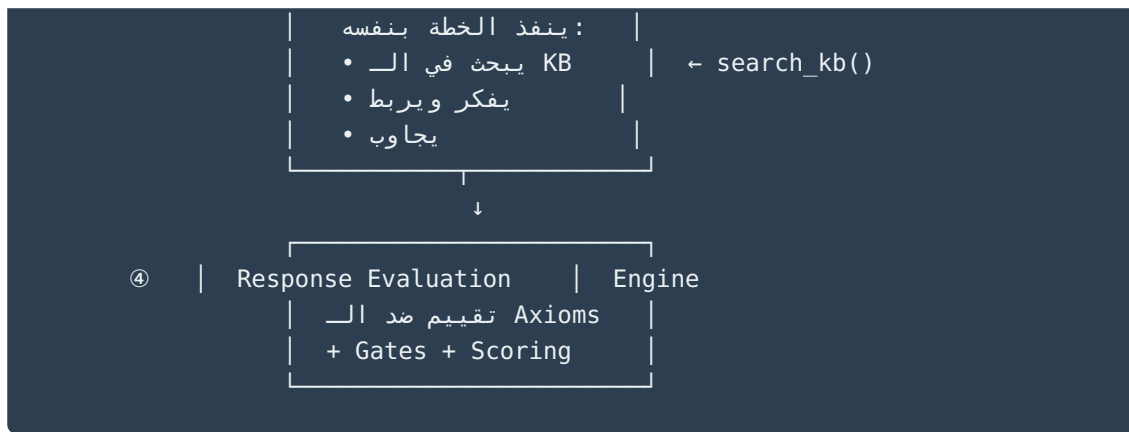
2. المشكلة الحالية

المقياس	حالياً	المشكلة
KB طريقة تمرير ال	كامل System prompt	المودل بيستلم كل حاجة — مش بيتعلم يبحث
context حجم ال	(67 entry) حرف ~28K	(~600K+) مش هيشيل 23 حلقة
توجيه التفكير	مفیش	المودل بيجاوب من غير منهج تفسيري
التحقق	مفیش	للإجابة verification مفیش
كمصدر ال KB	المصدر الوحيد	المفروض يكون مصدر مساعد مش بديل

3. Engine-Guided Reasoning Pipeline: الحل

3.1 الكامل Flow ال





ينفذ الخطة ويبحث **LLM** يبني الخطة ويقيّم النتيجة (المعلّم) - الـ **Engine** - الـ **المبدأ الأساسي**
أدوات بحث في يد المودل (المكتبة) **KB** بنفسه (الطالب) - الـ

تفصيل كل مرحلة 3.2

4. المرحلة ①: Query Analyzer (محلل السؤال)

الوظيفة:

يحلل سؤال المستخدم ويستخرج: - الآيات المذكورة صراحة وضمنياً - المواضيع والمفاهيم - نوع (يستخدم reasoning template يحدد أي) السؤال

الـ Output:

```

@dataclass
class QueryAnalysis:
    original_query: str
    verse_refs: list[str] # ["6:1", "6:2", "6:3", "6:4", "6:5"]
    topics: list[str] # ["السنة الإلهية", "الوعيد"]
    query_type: QueryType # أحد الأنواع التالية
    search_keywords_ar: list[str] # كلمات بحث
    needs_external_knowledge: bool # KB هل محتاج معرفة برا الـ

class QueryType(Enum):
    TAFSIR = "tafsir" # تفسير آية أو مجموعة آيات
    VERSE_LINK = "verse_link" # ربط بين آيات
    ISTINBAT = "istinbat" # استنباط ودروس
    COMPARISON = "comparison" # مقارنة بين سور أو مواضع
    SEERAH_LINK = "seerah_link" # ربط بالسيرة
    GENERAL = "general" # سؤال عام
    
```

التنفيذ:

- **المرحلة الأولى:** Regex + Rules (سريع، deterministic)
- **Fallback:** LLM call المعقدة
- مثال: "إيه علاقة أول 4 آيات بالآية 5" → `verse_refs=[6:1..6:5]`,
`query_type=VERSE_LINK`

5. (بناء خطة التفكير) Reasoning Plan Builder ② المرحلة

الوظيفة:

Axioms ال - (Query Analyzer من ال) للمودل بناءً على: - نوع السؤال **خطة تفكير كاملة** يبني المناسب **Reasoning Template** - (بوابات التحقق) **Four Gates** ال - (المسلّمات الثابتة)

المودل هو الذي ينفذ الخطة ويبحث بنفسه — **KB أدوات البحث في ال** الخطة بتتبع للمودل مع

5.1 في سياق التفسير Axioms ال

بترجم لقواعد تفسيرية Engine الموجودة في ال Axioms ال

Axiom	التطبيق التفسيري
Transcendence Necessity	القرآن مصدر متعالي — التفسير يرجع للنص أولاً مش للرأي البشري
Final Court Necessity	الوعد والوعيد في القرآن حقيقي — يجب ربطه بتحقيقه التاريخي
Design vs. Accident	ترتيب الآيات مقصود — كل آية في مكانها لحكمة
The Network Effect	كل آية مرتبطة بما قبلها وبعدها وبالسورة ككل — لا تفسير بمعزل

5.2 في سياق التفسير Four Gates ال

Gate	التطبيق التفسيري
Source-Integrity	التمز بالنص القرآني والحديث الصحيح — لا تحرف ولا تختزل
Structural-Consistency	الربط بين الآيات لازم يكون منطقي ومتسق — مش عشوائي
Mediation-Zeroing	الرأي البشري ليس حجة — ارجع للمصادر الموثوقة

Gate	التطبيق التفسيري
Origin-Aware	المرجعية هي الوحي — مش الفلسفة أو الثقافة

5.3 Reasoning Plan

كامل وبيعتها للمودل plan يبيني Engine الـ

```
@dataclass
class ReasoningPlan:
    query_analysis: QueryAnalysis
    axiom_guidelines: list[str]      # مترجمة للسياق Axioms قواعد من الـ
    gate_checks: list[str]          # بوابات التحقق اللي المودل يلتزم بيها
    reasoning_template: str          # خطوات التفكير حسب نوع السؤال
    kb_tools: list[ToolDefinition]   # أدوات البحث المتاحة للمودل
    output_format: str              # شكل الإجابة المطلوب
```

5.4 KB Tools (أدوات البحث المتاحة للمودل)

المودل بيستلم الأدوات دي ويستخدمها بنفسه أثناء التفكير

Tool 1: `search_kb_by_verse(verse_ref)`

```
# المرتبطة edges يبحث بالآية المركزية ويرجع كل الـ
search_kb_by_verse("6:5")
→ [6:112, 6:123, ...] الإلهية السنة , مسعود ابن حديث , بدر يوم ]
```

Tool 2: `search_kb_by_topic(topic)`

```
# (semantic search) يبحث بالموضوع
search_kb_by_topic("محاجة المشركين")
→ [edges المركزية الآيات كل من بالمحاجة مرتبطة ]
```

Tool 3: `search_kb_by_relation(verse_ref, relation_type)`

```
# يبحث عن نوع علاقة محدد
search_kb_by_relation("6:5", "LINKED_HADITH")
→ [الجزور سلا حديث , مسعود ابن حديث]
```

Tool 4: `get_verse_context(verse_ref, range=5)`

```
# يجب الآيات المحيطة بالآية + تفسيرها
get_verse_context("6:5", range=3)
→ [متاحة تفاسير + 6:2, 6:3, 6:4, 6:5, 6:6, 6:7, 6:8]
```

5.5 Reasoning Templates (المدمجة مع الـ Axioms)

Template: تفسير آية (TAFSIR)

خطة التفكير

المسلمات (Axioms):

- (Design) ترتيب الآيات مقصود – لماذا جاءت هذه الآية هنا؟
- (Network Effect) كل آية مرتبطة بسياقها – لا تفسر بمعزل

بوابات الجودة (Gates):

- □ Source-Integrity: هل استندت للنص القرآني والحديث الصحيح؟
- □ Structural-Consistency: هل الربط بين الآيات متسق ومنطقي؟
- □ Mediation-Zeroing: هل تجنبنا الرأي البشري بدون دليل؟
- □ Origin-Aware: هل المرجعية هي الوحي؟

خطوات التنفيذ:

1. ابحث عن الآية في الـ KB: `search_kb_by_verse("{verse_ref}")`
2. حدد الموضوع المركزي للآية
3. ابحث عن السياق: `get_verse_context("{verse_ref}")`
4. ابحث عن الروابط: `search_kb_by_relation("{verse_ref}", "LINKED_VERSE")`
5. ابحث عن الأحاديث: `search_kb_by_relation("{verse_ref}", "LINKED_HADITH")`
6. استخرج السنة الإلهية أو القاعدة الكلية
7. استنبط الدروس

القواعد:

- KB أجب من معرفتك الشاملة + ما تجده في الـ
- مصدر مساعد – لو مالقيت فيه أجب من معرفتك KB الـ
- لما تنقل من الشيخ أحمد السيد وضح ذلك

Template: ربط آيات (VERSE_LINK)

خطة التفكير

المسلمات:

- (Design) ترتيب الآيات مقصود
- (Network Effect) الربط بين الآيات جزء من فهم المعنى

بوابات الجودة:

- □ Source-Integrity: التزم بالنص
- □ Structural-Consistency: الربط منطقي ومتسق

خطوات التنفيذ:

1. لكل آية (search_kb_by_verse("{verse_ref}")) ابحث عن كل آية.
2. حدد الموضوع المشترك
3. تتبع التسلسل المنطقي
4. search_kb_by_topic("{topic}") ابحث عن الموضوع
5. حدد نقطة التحول بين المواضيع
6. استخرج العلاقة البنيوية

Template: مقارنة (COMPARISON)

خطة التفكير

المسلمات:

- (Design) القرآن يتناول المواضيع من زوايا مختلفة حسب السياق
- (Network Effect) التكرار في القرآن ليس تكراراً - كل موضع له حكمة

خطوات التنفيذ:

1. search_kb_by_topic("{topic}") عن الموضوع KB ابحث في الـ
2. حدد كيف تناولته سورة الأنعام
3. قارن بسور أخرى من معرفتك
4. حدد النمط المشترك والاختلاف
5. استنبط الحكمة من الاختلاف

(SEERAH_LINK و ISTINBAT مشابهة لـ Templates)

6. المرحلة ③: LLM Execution (تنفيذ المودل)

الوظيفة:

وينفذ بنفسه KB أدوات الـ + الكامل Reasoning Plan المودل يستلم الـ

يستلم المودل:

- (reasoning template + axioms + gates) التفكير خطة
- (search_kb_by_verse, search_kb_by_topic, ...) البحث أدوات
- الأصلي السؤال

ينفذ المودل:

- KB (tool calls) الـ في يبحث
- (الخطة بتوجيه) ويربط يفكر
- gates (self-check) الـ من يتحقق
- يجاوب

ال System Prompt:

أنت عالم متخصص في تفسير القرآن الكريم.

لديك أدوات بحث في قاعدة معرفة تفسيرية (من دروس الشيخ أحمد السيد).
استخدم هذه الأدوات أثناء تفكيرك للبحث عن مصادر تُثري إجابتك.

Engine الخطة الكاملة من ال ← {reasoning_plan}

قواعد:

1. اتبع خطوات التفكير المطلوبة بالترتيب.
2. استخدم أدوات البحث للعثور على مصادر مرتبطة.
3. KB أجب من معرفتك الشاملة + ما تجده في ال.
4. مصدر مساعد - لو مالمقيتش أجب من معرفتك KB ال.
5. المذكورة (Gates) التزم ببوابات الجودة.
6. لما تنقل من الشيخ أحمد السيد وضّح ذلك.

مثال تنفيذ (السؤال: "إيه علاقة أول 4 آيات بالآية 5")

المودل يفكر:

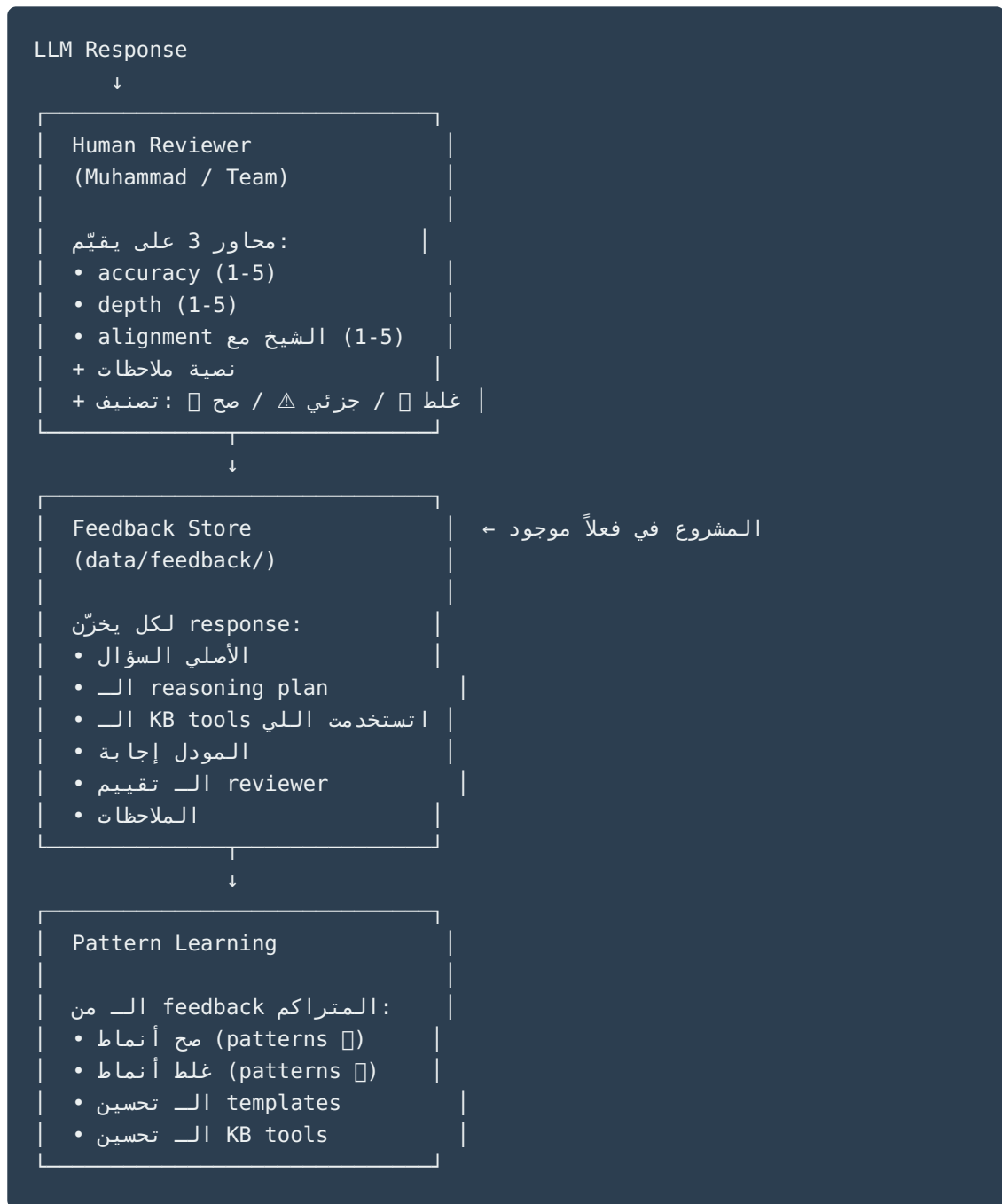
- الخطة تقول: ابحث عن الآية 6:5 أولاً
- Tool call: search_kb_by_verse("6:5")
- نتيجة: [يوم بدر, حديث ابن مسعود, السنة الإلهية]
- الخطة تقول: ابحث عن السياق
- Tool call: get_verse_context("6:5", range=4)
- نتيجة: [آيات 6:1-6:8 + تفاسير]
- الخطة تقول: ابحث عن العلاقات
- Tool call: search_kb_by_relation("6:5", "LINKED_VERSE")
- نتيجة: [6:123, 6:112]
- Gates الخطة تقول: تحقق من ال
- ✓ استندت للآيات والحديث: Source-Integrity
- ✓ الربط منطقي: Structural-Consistency
- ...دلوقتي أجاب

7. المرحلة ④: Human Review + Feedback Loop

7.1 الفلسفة:

الأوتوماتيكي ييجي Engine evaluation بشري — أنتم اللي بتقيّموا الإجابات. ال **حالياً** التقييم لما تتجمّع بيانات كفاية من تقييماتكم **لاحقاً**.

7.2 JI Flow:



7.3 JI Feedback Schema:

```
@dataclass
class TafsirFeedback:
    # === Input ===
    question: str
    query_analysis: QueryAnalysis
    reasoning_plan: ReasoningPlan
    kb_tools_used: list[dict]
    kb_results: list[dict]
    # السؤال الأصلي
    # تحليل السؤال
    # الخطة التي اتبعت للمودل
    # التي المودل عملها tool calls الـ
    # KB نتائج البحث من الـ
```

```

# === ال Output ===
llm_response: str          # إجابة المودل

# === ال Human Review ===
reviewer: str              # من قِـم
accuracy: int              # الإجابة صح؟ - 1-5
depth: int                 # فيها تفصيل وتحليل؟ - 1-5
alignment: int             # توافق مع منهج الشيخ؟ - 1-5
verdict: str               # "correct" / "partial" / "wrong"
notes: str                 # ملاحظات حرة

# === Pattern Tags ===
good_patterns: list[str]   # إيه اللي المودل عمله صح
bad_patterns: list[str]    # إيه اللي المودل عمله غلط
# مثال good: ["حديث ابن مسعود صح"] استخدم "ربط تاريخي دقيق", "استخدم
# مثال bad: ["مذكرش مصدر الحديث", "تجاهل السنة الإلهية"]

```

7.4 بيحسن النظام Feedback كيف ال

Reasoning Templates تحسين ال

يقول feedback لو أغلب ال
 □ "المودل مش بيربط بالسيرة كويس"
 → TAFSIR template: نضيف في ال
 "تأكد من ربط الآية بالحدث التاريخي المرتبط بها"

KB Tools تحسين ال

يقول feedback لو أغلب ال
 □ "search_kb_by_relation المودل ما استخدمش"
 → نضيف في الخطة:
 "search_kb_by_relation(verse, 'LINKED_HADITH') البحث عن الأحاديث المرتبطة"

Pattern Library بناء

المتراكم feedback من ال
 □ Patterns (يتكرروا) صح:
 - "لما الشيخ يذكر آية وعيد → ربطها بتحقيق تاريخي"
 - "لما يذكر قصة نبي → ربطها بموقف النبي ﷺ"
 - "لما يشرح حكم → رجع للسنة الإلهية"
 □ Patterns (يتفادوا) غلط:
 - "تفسير الآية بمعزل عن سياقها"

- "ذكر حديث بدون تخريج"
- "خلط بين آراء المفسرين والنص القرآني"

الهدف طويل المدى:

automated evaluator بتقييمات بشرية → نقدر نبني **100+ feedback entry** لما تتجمع قبل ما pre-screening الصح والغلط - يقدر يعمل patterns يحاكي حكمكم: - يتدرب على الـ يوصلكم - بس القرار النهائي يفضل بشري

الموجود FeedbackStore التكامل مع الـ 7.5:

— `submit(feedback)` - موجود فعلاً في المشروع ويدعم `store/feedback_store.py` الـ
`list_all()` - جلب تقييمات لإجابة معينة — `get_by_verdict(verdict_id)` - تخزين تقييم
 — Ratings: `agree` / `disagree` / `partial` / `flag` - كل التقييمات —
 الـ `TafsirFeedback` schema هنوسّعه ليدعم الـ pattern tags الجديد مع الـ

التكامل مع البنية الحالية 9.

الملفات الموجودة:

الدور الجديد	الدور الحالي	الملف
Tafsir KB بحث في +	Quran/Hadith/Fiqh بحث	<code>kb/retriever.py</code>
semantic search يخدم	MiniLM embeddings	<code>kb/embeddings.py</code>
graph retrieval يخدم	Graph traversal	<code>kb/knowledge_linker.py</code>
tafsir edges يخزن	Graph storage	<code>kb/graph/store.py</code>
Response Verifier يخدم	Scan→Mirror→Verdict	<code>engine/pipeline.py</code>
+ Reasoning Templates	Engine prompts	<code>engine/prompts.py</code>
+ Tafsir reasoning chains	Guided Reasoning	<code>engine/chains/</code>

الملفات الجديدة:

```
src/al_furqan/
├── kb/
│   └── tafsir/
│       ├── __init__.py
│       ├── query_analyzer.py      # تحليل السؤال ①
│       ├── kb_tools.py           # أدوات البحث (search_kb_by_verse, etc.)
│       ├── vector_store.py       # semantic search للـ embeddings تخزين
│       └── tool_executor.py      # من المودل tool calls ينفذ الـ
├── engine/
│   └── tafsir/
│       ├── __init__.py
│       ├── reasoning_plan_builder.py # بناء خطة التفكير ② (Axioms + Gates)
│       ├── reasoning_templates.py  # حسب نوع السؤال Templates
│       ├── response_evaluator.py  # Axioms + Gates تقييم ضد ④
│       └── correction_loop.py     # Self-correction فشل عند Gate
```

10. Data Pipeline

Engine-Guided RAG: من الحلقة للـ

```
YouTube Episode
  ↓
[Whisper Transcription]      ← موجود
  ↓
lesson_XX_transcript.json
  ↓
[Transcript Chunker]        ← موجود
  ↓
[Relationship Extractor]     ← موجود (prompt معدّ)
  ↓
proposed_edges.db           ← موجود
  ↓
[Human Review]              ← موجود (confirm/reject)
  ↓
```

NEW: Indexing Pipeline

1. Embed كل edge (reasoning + sheikh words)
 2. vector store في نّخز
 3. graph store الـ نّحد
 4. reasoning patterns استخراج (الشيخ تفكير أنماط)
- جديد ←

Reasoning Pattern Extraction (استخراج أنماط التفكير):

"تفكير الشيخ: - "لما الشيخ يفسر آية وعيد → يربطها بتحقيق تاريخي **أنماط** نقدر نستخرج KB من الـ
لما يذكر قصة نبي → يربطها بموقف النبي ﷺ المشابه" - "لما يشرح حكم → يرجع للسنة الإلهية" -
"الكلية"

وتخليها أدق مع الوقت Reasoning Templates هذه الأنماط تغذي الـ

11. خطة التنفيذ (Sprints)

Sprint 1: Query Analyzer + KB Tools (1 أسبوع)

- [] `query_analyzer.py` — تحليل السؤال (regex + rules + LLM fallback)
- [] `kb_tools.py` — 4 أدوات: `search_kb_by_verse`, `search_kb_by_topic`, `search_kb_by_relation`, `get_verse_context`
- [] `tool_executor.py` — ويرجع النتائج tool calls ينفذ الـ
- [] `vector_store.py` — embedding + FAISS للـ semantic search
- [] Tests: 10+ test cases لكل tool
- [] اختبار يدوي: المودل يقدر يستخدم الأدوات صح

Sprint 2: Reasoning Plan Builder + Templates (2 أسبوع)

- [] `reasoning_plan_builder.py` — Axioms + Gates يبنى خطة التفكير من Template
- [] `reasoning_templates.py` — 5 templates مدمجة مع Axioms و Gates
- [] لسياق تفسيري (كقواعد واضحة) Axioms + Gates ترجمة الـ
- [] Integration: المودل يستلم الخطة + الأدوات وينفذ بنفسه
- [] Tests: مع 3+ أسئلة template كل
- [] **A/B test**: full-KB vs Engine-Guided (alignment + depth + reasoning quality)

Sprint 3: Response Evaluator + Gate Scoring (3 أسبوع)

- [] `response_evaluator.py` — 4 Gates + KB usage + reasoning steps
- [] Scoring system (gate scores + bonuses + penalties)
- [] `correction_loop.py` — self-correction عند فشل gate (max 3 محاولات)
- [] Integration مع Scan→Mirror→Verdict pipeline
- [] Tests: evaluation accuracy tests

Sprint 4: Full Pipeline + Benchmark (4 أسبوع)

- [] End-to-end pipeline: Query → Plan → LLM+Tools → Evaluate
- [] Full benchmark: 12+ 3 × modes (zero-shot / full-KB / Engine-Guided)
- [] Measure: accuracy, depth, alignment, gate scores, KB tool usage, latency
- [] Reasoning pattern extraction (أولي)
- [] Documentation update + PDF

12. معايير النجاح

المقياس	Full-KB (حالي)	Engine-Guided (المطلوب)
Alignment مع الشيخ	~4.8/5	≥ 4.8/5 (يحافظ أو يتحسن)
Depth (عمق التفكير)	~4.9/5	≥ 5.0/5 (يتحسن بالتوجيه)
Context size	حرف 28K+	حرف ≤ 8K
Scalability	□	□ (يشيل أي عدد حلقات)
Reasoning structure	عشوائي	منظم بخطوات
Source attribution	ضعيف	(يوضح لما ينقل من الشيخ) واضح
External knowledge	KB محدود بالـ	(مساعد مش حد KB) حر

13. المخاطر والحلول

المخاطر	الاحتمال	الحل
مش مناسبة لكل أنواع templates الـ الأسئلة	متوسط	تحسين + fallback ≤ Template GENERAL مستمر
template يختار Query Analyzer الـ غلط	متوسط	لمراجعة القرارات LLM fallback + logging
ضعيف بالعربي Semantic search	متوسط	CamelBERT بدل MiniLM + fine-tuning
المودل يتجاهل خطوات التفكير	منخفض	Response Verifier + يرصد ده re-prompt
retrieval فقدان معلومة بسبب الـ	منخفض	Fallback: وسّع confidence لو البحث المنخفض

14. التطور المستقبلي

Phase 2 (بعد الـ 4 sprints):

- **Adaptive Templates** — أفضل لأي template أي feedback يتعلم من الـ Engine الـ نوع سؤال
- **Reasoning Pattern Library** — مكتبة أنماط تفكير مستخرجة تلقائياً من كل الحلقات
- **Multi-Scholar Support** — لشيوخ ومفسرين تانيين pipeline نفس الـ
- **Student Progress Tracking** — يتتبع مستوى المستخدم ويكيّف عمق الإجابة

Phase 3 (طويل المدى):

- من أنماط الشيخ المكتشفة **جديدة templates** يولّد الـ Engine الـ
- **Fine-tuning** — مودل متخصص fine-tune لـ benchmark data نستخدم الـ
- **Cross-Surah Reasoning** — من سور متعددة KBs لما تتوفر

15. Capacity Planning & Performance

15.1 تحليل أداء كل مرحلة 15.1

المرحلة	العملية	الوقت المتوقع	الموارد
① Query Analyzer	Regex/Rules	~5ms	CPU فقط
② KB Retriever (Verse)	SQLite query	~10ms	Disk I/O بسيط
② KB Retriever (Semantic)	Embedding + FAISS search	~50-100ms	RAM (embeddings محملة)
② KB Retriever (Graph)	Graph traversal	~20ms	RAM
③ Reasoning Architect	Template selection + formatting	~5ms	CPU فقط
④ LLM Generation	API call (Qwen/DashScope)	ثانية 5-15 ~ ⚠️	JI bottleneck
⑤ Response Verifier	Rule checks + optional LLM	ثانية 1-8 ~	إضافي LLM call

JI Bottleneck ثانية/queries مجتمعة → تستحمل آلاف الـ ms أقل من 200: ③-① المراحل
LLM API مرحلة ④ الـ: **الوحيد**

15.2 حدود LLM API (DashScope/Qwen)

المقياس	القيمة
LLM وقت رد الـ	5-15 ثانية/query
Rate limit	~60 RPM (request/minute)
Concurrent queries	في نفس الوقت 4-5 ~
عند تجاوز الحد	429 Too Many Requests (timeout مش crash)

سيناريوهات التحمل 15.3

السيناريو	الحالة	وقت الاستجابة	ملاحظات
متزامنين 1-5 users	مريح	ثانية 5-15	الوضع الطبيعي
متزامنين 10-20 users	بطء	ثانية 30-60	Queue بيطول
متزامنين 50+ users	مشكلة	فشل/timeout	Rate limit، queries بعض الـ بتفشل

نقاط الفشل المحتملة والحلول 15.4

المشكلة	السبب	الحل
Out of Memory	كثير في الـ Embeddings RAM	Lazy loading + FAISS memory-mapped
SQLite lock	كتابة وقراءة متزامنة	WAL mode (موجود) → لاحقاً PostgreSQL
LLM timeout	بطيء تحت الضغط API	Retry with backoff + request queue
Context too large	retrieval → سؤال معقد كبير	Hard cap (8K حرف max)
Embedding model crash	صغير RAM مودل كبير على	MiniLM (خفيف) → CamelBERT لاحقاً

15.5 خطة التوسع (Scaling Phases)

Phase 1: MVP (5-1 users حالي —)

SQLite + DashScope API + Single Server
أو ~\$10/شهر (free tier) التكلفة: ~\$0

- testing والـ development كفاية للـ
- إضافي infrastructure مفيش

Phase 2: Small Scale (10-50 users)

+ Request Queue (Redis/Celery)
+ LLM Response Cache
+ Multiple API Keys
التكلفة: ~\$50-100/شهر

- **Request Queue:** وتنفيذ بالترتيب — المستخدم يستنى queue تنتظ في queries ال error بدل ما يجيله
- **Response Cache:** بدون cache نفس السؤال (أو سؤال شبهه) → نفس الإجابة من ال LLM call
- **Multiple API Keys:** rate limit نضاعف ال key على أكثر من load نوزع ال

Phase 3: Medium Scale (50-500 users)

+ Local LLM (Qwen على GPU)
+ PostgreSQL بدل SQLite
+ Horizontal Scaling (multiple workers)
التكلفة: ~\$200-500/شهر (GPU server)

- **Local LLM:** بس GPU بيتحدد بال throughput ال → rate limit لا
- دقيقة/queries: ~20-30 (A100/H100): واحد GPU على
- **PostgreSQL:** SQLite أكثر من concurrent connections يستحمل
- **Workers:** horizontal scaling → كامل pipeline بيخدم worker كل

Phase 4: Large Scale (500+ users)

+ Fine-tuned Model (أصغر وأسرع)
+ Edge Caching للأسئلة الشائعة
+ Distributed Inference (multiple GPUs)
+ CDN لل static responses
التكلفة: ~\$1000+/شهر

- **Fine-tuned model:** من أسرع 5-10 → tafsir متدرب على ال (7B-14B) مودل أصغر مودل عام كبير
- **Edge caching:** بيترد عليها فوراً (80% traffic من ال) الأسئلة الشائعة
- **Distributed inference:** Multiple GPUs → 100+ queries/دقيقة

15.6 Capacity ملخص ال

المرحلة	متزامنين Users	دقيقة/Queries	التكلفة/شهر
Phase 1 (MVP)	1-5	~4-5	~\$0-10
Phase 2 (Queue+Cache)	10-50	~15-20	~\$50-100
Phase 3 (Local LLM)	50-500	~30-60	~\$200-500
Phase 4 (Distributed)	500+	100+	~\$1000+

cache يحمي من الفشل، وال queue ال **slow down** هي — **crash** مش هي النظام **القاعدة** بنضيفها لما نحتاجها — مش من الأول phase يقلل الحمل. كل

16. الخلاصة

التقليدي RAG وال approach الفرق بين هذا ال

تقليدي RAG	Engine-Guided Reasoning
يجيب معلومات	يعلّم طريقة تفكير
المصادر = الإجابة	للتفكير مادة خام = المصادر
المودل ينسخ	يفكر بتوجيه المودل
Static retrieval	Dynamic reasoning plan
One-size-fits-all	Template per question type

بيعمل زي ما الشيخ أحمد السيد بيعمل بالظبط: - مش يقول "احفظ التفسير" - يقول Engine ال "شوف الآية دي... ربطها بدي... لاحظ السنة الإلهية... دلوقتي استنبط"